Department of Computer Science and Engineering
University of Moratuwa

CS4533 – Concurrent Programming

Lab 02: Producer-Consumer, Reader-Writer & PThreads

Module:	CS4533 – Concurrent Programming
Lab:	Lab 02 – Producer-Consumer, Reader-Writer & PThreads
Duration:	2 hours (in-class)
Topics:	Condition variables, producer-consumer, reader-writer, starvation

Prerequisites

- Completed Lab 01 (threads, mutexes)
- Understanding of pthread_mutex_lock() and pthread_mutex_unlock()
- Compile all exercises with: gcc -pthread -O0 -o output source.c

Learning Objectives

By the end of this lab, you should be able to:

- Use POSIX condition variables (pthread_cond_wait, pthread_cond_signal, pthread_cond_broadcast)
- Implement the bounded-buffer producer-consumer pattern
- Implement the reader-writer synchronization pattern
- Identify and reason about starvation and fairness in concurrent programs

Exercise 1: Condition Variables — Ping Pong

Background

A condition variable allows a thread to sleep until a particular condition is met, rather than busy-waiting. Combined with a mutex, it provides an efficient mechanism for thread coordination. The key functions are:

- pthread_cond_wait(&cond, &mutex) — atomically releases the mutex and sleeps until signalled
- pthread_cond_signal(&cond) — wakes one waiting thread
- pthread_cond_broadcast(&cond) — wakes all waiting threads

Task

Write a C program with two threads: one prints "ping" and the other prints "pong". They must strictly alternate, producing the output: ping, pong, ping, pong, ... for N rounds (N provided via command line). Each thread prints exactly N times.

Requirements

- Use one mutex and one or two condition variables (your choice) to enforce alternation.
- Use a shared turn variable to track whose turn it is.
- The program must produce perfectly alternating output every time, starting with "ping".
- Test with N = 5, 10, and 20.

Starter Code

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int turn = 0;  // 0 = ping's turn, 1 = pong's turn
int N;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

void *ping(void *arg) {
    for (int i = 0; i < N; i++) {
        // TODO: Wait until it's ping's turn
        // TODO: Print "ping"
        // TODO: Set turn to pong and signal
    }
    return NULL;
}

void *pong(void *arg) {
    for (int i = 0; i < N; i++) {
        // TODO: Wait until it's pong's turn
        // TODO: Print "pong"
        // TODO: Set turn to ping and signal
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    N = atoi(argv[1]);
    pthread_t t1, t2;
    pthread_create(&t1, NULL, ping, NULL);
    pthread_create(&t2, NULL, pong, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

Questions

Q1.1: What happens if you remove the while loop around pthread_cond_wait() and use a plain if statement instead? Test it and explain the concept of spurious wakeups.

Q1.2: What happens if you use pthread_cond_broadcast() instead of pthread_cond_signal() in this exercise? Does it affect correctness? Why or why not?

Exercise 2: Producer-Consumer — Single Producer, Single Consumer

Background

The producer-consumer problem is a classic synchronization scenario. A producer generates data items and places them into a shared bounded buffer. A consumer removes items from the buffer and processes them. The challenges are: the producer must wait if the buffer is full, and the consumer must wait if the buffer is empty.

Task

Implement a bounded-buffer producer-consumer system with one producer thread and one consumer thread. The buffer is a circular array of fixed size (BUFFER_SIZE = 5). The producer generates integers 0 through N-1, and the consumer reads and prints each item.

Requirements

- Implement the buffer as a circular array with head, tail, and count variables.
- Use one mutex to protect the buffer and two condition variables: one for "buffer not full" and one for "buffer not empty".
- The producer must block (not busy-wait) when the buffer is full.
- The consumer must block when the buffer is empty.
- After the producer finishes, it places a special sentinel value (-1) to signal the consumer to stop.
- Test with N = 10 and N = 100. Verify that all items are consumed in order.

Starter Code

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int head = 0, tail = 0, count = 0;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t not_full  = PTHREAD_COND_INITIALIZER;
pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;

int N;  // number of items to produce

void put(int item) {
    // TODO: Wait while buffer is full
    // TODO: Add item to buffer at tail position
    // TODO: Update tail and count
    // TODO: Signal not_empty
}

int get() {
    // TODO: Wait while buffer is empty
```

```
    // TODO: Remove item from buffer at head position
    // TODO: Update head and count
    // TODO: Signal not_full
    // TODO: Return the item
}

void *producer(void *arg) {
    for (int i = 0; i < N; i++) {
        put(i);
        printf("Produced: %d\n", i);
    }
    put(-1);  // sentinel
    return NULL;
}

void *consumer(void *arg) {
    while (1) {
        int item = get();
        if (item == -1) break;
        printf("Consumed: %d\n", item);
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    N = atoi(argv[1]);
    pthread_t prod, cons;
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
    printf("All items consumed.\n");
    return 0;
}
```

Questions

Q2.1: What happens if you set BUFFER_SIZE = 1? Does the program still work correctly? How does it affect performance compared to BUFFER_SIZE = 5? Explain.

Q2.2: What is the purpose of using two separate condition variables (not_full and not_empty) instead of a single one? What problem could arise with a single condition variable?

Exercise 3: Producer-Consumer — Multiple Producers, Multiple Consumers

Background

Real systems typically have multiple producers and consumers sharing a single buffer. This introduces additional complexity: multiple threads may wake up simultaneously when signalled, and the buffer state may change between being signalled and actually executing. This is where the distinction between pthread_cond_signal and pthread_cond_broadcast becomes critical.

Task

Extend your Exercise 2 solution to support M producer threads and N consumer threads sharing a single bounded buffer (BUFFER_SIZE = 10). Each producer generates K items. Consumers stop when they receive a sentinel value.

Requirements

- Each producer generates items tagged with its producer ID: value = producer_id * 1000 + sequence_number (e.g., producer 2 generates 2000, 2001, 2002, ...).
- Each producer places a sentinel (-1) when it finishes. Consumers stop only after receiving M sentinels total (one from each producer).
- Print which thread produced and consumed each item, using a format like: "Producer 2: put 2005 | Consumer 1: got 2005"
- After all threads finish, print a summary: total items produced, total items consumed, and whether they match.
- Test with the following configurations:
  - Configuration A: M=3 producers, N=1 consumer, K=20 items each
  - Configuration B: M=1 producer, N=3 consumers, K=20 items
  - Configuration C: M=3 producers, N=3 consumers, K=20 items each

Questions

Q3.1: In which configuration (A, B, or C) did items get consumed fastest? Why? Present timing results for all three.

Q3.2: Why is pthread_cond_broadcast() generally preferred over pthread_cond_signal() in the multi-producer/multi-consumer case? Test with both and describe any issues you observe.

Q3.3: Is it possible for items from different producers to be consumed out of their original production order? Did you observe this? Explain why this is or isn't a problem.

Exercise 4: Reader-Writer Problem

Background

The reader-writer problem involves concurrent access to shared data where multiple readers can read simultaneously (since reading doesn't modify data), but writers require exclusive access. The challenge is implementing this asymmetric access pattern efficiently while avoiding starvation.

Task

Implement a reader-writer solution where R reader threads and W writer threads access a shared integer variable. Readers read the value, and writers increment it. Your implementation should allow multiple concurrent readers but give writers exclusive access.

Requirements

- Implement using mutexes and condition variables (not pthread_rwlock_t — you'll implement the logic yourself).
- Maintain a reader_count variable to track the number of active readers.
- Each reader: acquires read access, reads the shared value, sleeps for a short random duration (simulating work), releases read access. Repeats 5 times.

- Each writer: acquires write access, increments the shared value, sleeps for a short random duration, releases write access. Repeats 5 times.
- Print log messages with timestamps showing when readers and writers start and finish:

```
[  0.003s] Reader 2: started reading (value=5, active_readers=3)
[  0.005s] Reader 2: finished reading
[  0.006s] Writer 1: started writing (value 5 -> 6, active_readers=0)
[  0.008s] Writer 1: finished writing
```

- Test with: (a) R=5, W=1 (b) R=1, W=5 (c) R=5, W=5

Starter Code

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

int shared_value = 0;
int reader_count = 0;
int writer_active = 0;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t can_read  = PTHREAD_COND_INITIALIZER;
pthread_cond_t can_write = PTHREAD_COND_INITIALIZER;

struct timespec start_time;

double elapsed() {
    struct timespec now;
    clock_gettime(CLOCK_MONOTONIC, &now);
    return (now.tv_sec - start_time.tv_sec)
         + (now.tv_nsec - start_time.tv_nsec) / 1e9;
}

void start_read() {
    // TODO: Lock mutex
    // TODO: Wait while a writer is active
    // TODO: Increment reader_count
    // TODO: Unlock mutex
}

void end_read() {
    // TODO: Lock mutex
    // TODO: Decrement reader_count
    // TODO: If no more readers, signal writers
    // TODO: Unlock mutex
}

void start_write() {
    // TODO: Lock mutex
    // TODO: Wait while there are readers OR a writer is active
    // TODO: Set writer_active = 1
```

```
    // TODO: Unlock mutex
}

void end_write() {
    // TODO: Lock mutex
    // TODO: Set writer_active = 0
    // TODO: Signal readers and writers
    // TODO: Unlock mutex
}

void *reader(void *arg) {
    int id = *(int *)arg;
    for (int i = 0; i < 5; i++) {
        start_read();
        printf("[%7.3fs] Reader %d: reading (value=%d, active_readers=%d)\n",
               elapsed(), id, shared_value, reader_count);
        usleep(rand() % 100000);  // simulate work
        end_read();
        printf("[%7.3fs] Reader %d: done\n", elapsed(), id);
        usleep(rand() % 50000);   // think time
    }
    return NULL;
}

void *writer(void *arg) {
    int id = *(int *)arg;
    for (int i = 0; i < 5; i++) {
        start_write();
        int old = shared_value;
        shared_value++;
        printf("[%7.3fs] Writer %d: wrote (value %d -> %d)\n",
               elapsed(), id, old, shared_value);
        usleep(rand() % 100000);  // simulate work
        end_write();
        printf("[%7.3fs] Writer %d: done\n", elapsed(), id);
        usleep(rand() % 50000);   // think time
    }
    return NULL;
}
```

Questions

Q4.1: From your logs with R=5, W=5, identify at least one instance where multiple readers were active simultaneously. How can you confirm from the log that no writer was active during that time?

Q4.2: With R=5, W=1, does the single writer ever get to write? Approximately how long does it take compared to the R=1, W=5 configuration? This is a demonstration of potential writer starvation — explain why it happens.

Exercise 5: Writer-Priority Reader-Writer

Background

The solution in Exercise 4 gives readers priority — as long as readers keep arriving, writers may starve. A writer-priority solution ensures that once a writer is waiting, no new readers are admitted until the writer has finished.

Task

Modify your Exercise 4 solution to implement writer-priority. Add a waiting_writers counter. When a writer is waiting, new readers must block even if no writer is currently active.

Requirements

- Add an integer waiting_writers = 0 to track how many writers are queued.
- Modify start_read() to also block when waiting_writers > 0.
- Modify start_write() to increment waiting_writers before waiting, and decrement it after acquiring access.
- Test with R=5, W=5 and compare the logs with your Exercise 4 output.

Hints

```
void start_read() {
    pthread_mutex_lock(&mutex);
    while (writer_active || waiting_writers > 0) {  // <-- key change
        pthread_cond_wait(&can_read, &mutex);
    }
    reader_count++;
    pthread_mutex_unlock(&mutex);
}
```

Questions

Q5.1: Compare the logs from Exercise 4 and Exercise 5 with R=5, W=5. In which version do writers complete faster? Present average write completion times for both.

Q5.2: Is it now possible for readers to starve in your writer-priority solution? Under what conditions? Is there a design that avoids starvation for both readers and writers?

Submission Guidelines

- Submit 5 source files (.c), one per exercise, named as: Lab02_<IndexNumber1_IndexNumber2>_Ex<N>.c
- Submit one PDF report containing: your code output (terminal screenshots), answers to all questions, and analysis tables.
- The report should include log excerpts highlighting key behaviours (concurrent reads, writer starvation, etc.).
- Submit via Moodle before the deadline.

Marking Rubric

Component	Marks	Criteria
Ex 1: Ping Pong	5	Correct alternation, condition variable usage, Q1 answers
Ex 2: Single Producer-Consumer	10	Correct bounded buffer, blocking on full/empty, Q2 answers
Ex 3: Multi Producer-Consumer	10	Multiple P/C working, sentinel handling, timing comparison, Q3 answers
Ex 4: Reader-Writer	10	Concurrent reads, exclusive writes, timestamped logs, starvation analysis
Ex 5: Writer-Priority	5	Correct priority logic, comparison with Ex4, fairness discussion
Total	**40**	